

App Dev Project Report

1. Student Details

- **Name:** Rohit Kumar
 - **Roll Number:** 23f1000965
 - **Email:** 23f1000965@ds.study.iitm.ac.in
 - **About Me:** I am a diploma-level student and an enthusiastic learner in web development. I enjoy building practical full-stack applications and improving them through debugging and new feature integration.
-

2. Project Details

- **Project Title:** Trekking Management Application

Problem Statement

To design and build a role-based trekking management web application where Admin, Trek Staff, and Users (Trekkingers) can efficiently manage trek route registration, slot availability, registrations, automated notifications, and reporting workflows.

Approach

The project is developed using Flask for the API backend and Vue.js. SQLAlchemy is utilized for ORM operations on a local SQLite database. The system enforces role-based access control using JWT tokens. For high performance and background processing, Redis caching is implemented on frequently accessed trek lists, while Celery handles automated daily reminder emails, monthly summary reporting, and user-triggered booking history CSV exports.

3. AI/LLM Declaration

I used ChatGPT assistance for selected tasks such as API shaping suggestions, documentation drafting ,Celery configuration, and UI/caching debugging. The estimated AI/LLM usage is moderate (around 10-15%) and for productivity support. Final integration decisions, debugging, verification, and project-level implementation were performed manually.

4. Technologies and Frameworks Used Technology

/ Library Purpose

Technology / Library Purpose

Flask	Core backend web API framework
Flask-SQLAlchemy	SQL Object-Relational Mapper (ORM) for database interactions
Flask-JWT-Extended	Authentication and secure role-based API protection

Technology / Library Purpose

Flask-CORS	Cross-Origin Resource Sharing enablement for API
SQLite	Primary relational database
Redis	In-memory database used for API caching and Celery message broker/backend
Celery	Distributed task queue for executing asynchronous and scheduled batch jobs
MailHog	Local SMTP test server to intercept and review sent HTML/text emails
VueJS	Frontend framework for dynamic UI and SPA components
HTML / CSS / JS	Basic client-side markup, custom design/styling, and routing
Bootstrap 5	Responsive layout grid, modals, and style components

5. Database Schema

Main Tables

User (users):

Stores core credentials, contact details, role (ADMIN, STAFF, USER), and active status.

Trek (treks):

Stores trek routes, name, location, difficulty , duration, slots, status enum, and dates.

Booking (bookings):

Tracks registrations of trekkers to treks, including booking date, booking status, and payment status (with a unique constraint on (user_id, trek_id)).

StaffAssignment (staff_assignments):

Relates staff members to treks they guide (with a unique constraint on (staff_id, trek_id)).

ExportJob (export_jobs):

Logs background Celery CSV export status and file paths.

NotificationLog (notification_logs):

Records historical details of daily reminders and monthly report emails.

Relationships

- One-to-Many: User (STAFF) -> Trek (assigned guide)
- One-to-Many: User (USER) -> Booking (bookings made)
- One-to-Many: Trek -> Booking (registered participants)
- One-to-Many: User -> ExportJob
- One-to-Many: User -> NotificationLog
- Many-to-Many: Staff (users with role STAFF) <-> Trek (treks via StaffAssignment)

6. API Resource Endpoints (Representative)

Auth APIs

- POST /api/auth/register - Register trekker account
- POST /api/auth/login - Authenticate users
- GET /api/auth/profile - Fetch current user profile
- PUT /api/auth/profile - Edit user profile details

Admin APIs

- GET /api/admin/dashboard-stats - Fetch total stats for treks, bookings, users, and staff
- GET /api/admin/treks - Fetch all treks list
- POST /api/admin/treks - Create a new trek route
- GET /api/admin/treks/<trek_id> - Fetch specific trek details
- PUT /api/admin/treks/<trek_id> - Edit trek route details
- DELETE /api/admin/treks/<trek_id> - Delete trek route
- GET /api/admin/staff - Fetch list of all guides/staff members
- POST /api/admin/staff - Create a new guide/staff member account
- POST /api/admin/treks/<trek_id>/assign-staff - Assign staff to guides list
- GET /api/admin/trekkers - Fetch list of registered trekkers
- POST /api/admin/users/<user_id>/toggle-status - Toggle user login status
- GET /api/admin/bookings - Fetch all historical bookings
- POST /api/admin/trigger-daily-reminders - Manually trigger daily reminder scheduled job

- POST /api/admin/trigger-monthly-report - Manually trigger monthly report email

Trek Staff APIs

- GET /api/staff/treks - View assigned treks list
- GET /api/staff/treks/<trek_id>/participants - View trek participants roster
- PUT /api/staff/treks/<trek_id> - Edit slots, status, or complete/start trek status

Trekker (User) APIs

- GET /api/trekkers/treks - Get list of open treks (filtered, cached in Redis)
- POST /api/trekkers/bookings - Create a booking for a trek route
- GET /api/trekkers/bookings - View booking history log
- POST /api/trekkers/bookings/<booking_id>/cancel - Cancel a booked trek
- POST /api/trekkers/bookings/export - Trigger async export of booking history to CSV
- GET /api/tasks/<task_id>/status - Get status and download link of export task

YAML API Definition File:

Included separately in the submission ZIP as api.yaml.

7. Architecture and Implemented Features

Architecture Overview

backend/app.py

Application entry point. Handles server configuration, CORS, JWT extension setup, and REST API routes.

backend/models/init.py

SQLAlchemy database tables, constraints, relationships, and serialization formats.

backend/celery_app.py

Celery application creation and beat scheduler crontab intervals setup.

backend/tasks.py

Asynchronous jobs: Daily reminders (MailHog SMTP), Monthly summary reports (MailHog SMTP), and CSV history export.

backend/seed.py

Initialization script to programmatically build the database tables and seed default superuser credentials.

frontend/index.html

Single Page Application container utilizing bootstrap styles and loading client-side JavaScript.

frontend/

Folder containing the frontend app entry (app.js), components, router utilities, styles, and dashboard page views served as static assets.

8.Video Presentation**Drive link:-**

<https://drive.google.com/file/d/1QjeiXL9hHzFYxtHcmSqgrCERE93a0-Bx/view?usp=sharing>